Tantalus or MOCCa v2.0, a short guide to the input parameters

W. Ryssens* and M. Bender†
IPNL, Université de Lyon, Université Lyon 1, CNRS/IN2P3, F-69622 Villeurbanne, France

P.-H. Heenen‡
PNTPM, CP229, Université Libre de Bruxelles, B-1050 Bruxelles, Belgium
(Dated: 18th of August 2018)

I. INTRODUCTION; RUNNING THE CODE

For a succesful run of Tantalus, one needs, a) a compiled Tantalus.exe, b) a forces.param file containing the parameterization desired and c) a set of runtime data, to be provided to Tantalus on STDIN. This manual documents (in rather concise fashion) the structure of the forces.param file and the STDIN input for Tantalus. It does NOT document ALL of the possible options of the code, but only the ones that are of key importance to new users.

In short, the code can be run succesfully using

```
./Tantalus.exe < tant.data
```

where tant.data contains input, as formatted in section II. In the current working directory, a 'forces.param' file is present, formatted along the lines of section III, that contains the info of the asked-for parametrization.
For examples that can easily be run, please look in the tantalus_examples folder in this repository for a set of tutorial scripts.

II. THE RUNTIME INPUT

Tantalus reads input from STDIN, using the FORTRAN namelist machinery. The available namelists are indicated in Fig. 1; the ordering of the namelist is important. The most important keywords for every namelist are explained on the next page.
One of these namelists is special, the MomentConstraint namelist: it is repeatable. Every occurence of this namelist indicates to Tantalus that a constraint on a specific multipole moment $\langle \hat{Q}_{\ell m} \rangle$ should be included in the calculation. A calculation with a constraint on X multipole moments, should thus include X separate MomentConstraint namelists in the input data.

```
&nucleus/
&mesh/
&func/
&pairing/
&evolution/
&scfiteration/
&wfs/
&IO/
&MomentParam/
&MomentConstraint/
```

FIG. 1. List of namelist inputs, in the correct order.

* w.ryssens@ipnl.in2p3.fr
† bender@ipnl.in2p3.fr
‡ phheenen@ulb.ac.be

```
-------------------------------------------------------------------------------
&nucleus
neutrons   = Number of neutrons. Can be non-integer.
protons    = Number of protons. Can be non-integer.
/
-------------------------------------------------------------------------------
&mesh
nx/y/z     = number of points in x/y/z direction.
dx         = mesh discretisation (in fm).
/
-------------------------------------------------------------------------------
&func
name_param  = name of the parametrization, needs to match name on forces.param.
/
-------------------------------------------------------------------------------
&pairing
Type                  = ‘HF’, ‘BCS’ or ‘HFB’.
Cuttype               = Symmetric Fermi cutoff (1) or cosine cutoff (2).
Cutneutron, Cutproton = energy-distance around Fermi energy for cutoff.
FermiSolver           = Use a secant solver (’SECANT’) or a solver based on
                        Brent’s method (’BRENT’, default) to find the Fermi
                        energy for the HFB case. The first is simpler,
                        but the second is more stable.
HFBmix                = Proportion of pairing matrices rho and kappa to update
                        from one iteration to the next. Default value=1.0,
                        which corresponds to no dampening of the evolution of
                        the pairing subproblem.
BlockType             = No blocking (0), True blocking (1&2),
                        EFA based on indices(3), EFA based on lowest energy(4).
BlockNumber           = Number of quasiparticles to excite.
/
-------------------------------------------------------------------------------
&Indices
!             NOTE: this namelist is only read if BlockNumber in the
!                   &pairing/ namelist is nonzero.
BlockIndices = Blocknumber integers. These are the indices of the states in the
               Hartree-Fock basis, for which the code will attempt to construct
               quasiparticle excitations with large overlap with them.
BlockLowest  = Set of ’a2’ formatted strings, of length BlockNumber.
               ’n+’,’n-’ : lowest quasi-neutron of positive/negative parity
               ’p+’,’p-’ : lowest quasi-proton of positive/negative parity
               ’n0’,’p0’ : lowest quasi-neutron/proton of either parity.
               Note that all of these can be combined. The input
                ’n+’, ’n+’, ’p0’, ’p0’
               will result in the code trying to construct the lowest state
               with two-quasi-neutrons of positive parity and two quasi-protons
               of whatever parity.
/
-------------------------------------------------------------------------------
&evolution
maxiter     = maximum number of mean-field iterations.
printiter   = Large printouts after every multiple of this value.
/
-------------------------------------------------------------------------------
&scfiteration
preconfactor   = Factor beta in the preconditioning of the potential F_I_I.
                 Higher values result in more stable, but slower convergence.
                 The default value, 1.0, is in my (W.R.) experience really good.
```

```
                Only if your convergence problems are due to the explosion
                of mean-field potentials will changing this parameter help you.
/
-------------------------------------------------------------------------------
&wfs
nwn            = number of neutron wavefunctions.
                 Needs to match the input file, if present.
nwp            = number of proton wavefunctions.
                 Needs to match the input file, if present.
/
-------------------------------------------------------------------------------
&IO
Inputfilename  = name of the .wf file Tantalus will read.
                 If this is 'INIT', Tantalus will not read any file and initialize from scratch.
Outputfilename = name of the .wf file Tantalus will write.
BXLFIT         = string. Extra output will be written to the following file
                  '${BXLFIT}zXXXnYYY.out'
                 where XXX      = the number of protons  (forced to be integer)
                       YYY      = the number of neutrons (forced to be integer)
                       ${BXLFIT}= the value of the string.

                On this file is a single line with
                    Z, N, Total energy, Q20(n), Q20(p), Q22(n), Q22(p),        &
                 & Q(n), Q(p), G(n), G(p), <r^2_p>

                where E is the total energy calculated from the functional, the
                proton radius is calculated from the charge density, Q is the
                total deformation, and G is the triaxiality angle gamma.
                (Q and G are defined in Eq. 68 and 69 in
                   W. Ryssens, CPC 187, 175-194 (2015))
/
-------------------------------------------------------------------------------
&MomentParam
MoreConstraints = Logical. If .true., signals that the namelist &MomentConstraint/ will follow.
ContinueAll     = Logical. If .true., signal that the data of the constrained
                  multipole moments should be taken from the input .wf file, not
                  from STDIN.
/
-------------------------------------------------------------------------------
&MomentConstraint
l               = Integer, indicating the degree l of the multipole moment Qlm to constrain.
m               = Integer, indicating the degree m of the multipole moment Qlm to constrain.
Constraint      = Value to constrain <Qlm> to.
Iteration       = If zero (0), Tantalus will constrain the moment for the entire run.
                  If non-zero, the constraint will only be used for this number of iterations.
                  Very useful for breaking self-consistent symmetries.
MoreConstraints = Logical. If .true., signals that more &MomentConstraints will follow.
                           If .false., signals that this is the last &MomentConstraint.
Multfromfile    = Logical. If .true., start the calculation with the Lagrange multiplier on file.
/
-------------------------------------------------------------------------------
```

III. THE STRUCTURE OF A PARAMETERIZATION (.PARAM) FILE

The forces.param file should (for the current version of the code) only contain one namelist SKF. The content of this file varies, depending on the choices made at compilation time by Hephaestos. We will thus describe here only the particular keywords in the public version.

```
&skf
name     = "SLy4"           ! Name of the parameterization.
                            ! This needs to match name_param in the input data
func_file= 'NLO.func'       ! Name of the functional file used for compilation.
t0       =  -2488.913,      ! t0 parameter of the Skyrme parameterization
t1       =    486.818,      !
t2       =   -546.395,      !
t3       =  13777.0,        !
x0       =      0.834,      !
x1       =     -0.344,      !
x2       =     -1.0,        !
x3       =       1.354,     !
sigma    =      0.1666667   ! Exponent of the density dependence.
wso      =    123.0,         ! Spin-orbit strength.
wsoq     =    123.0          !
CT0      =      0.0          ! Coupling constant (isoscalar) of the J^2 terms
CT1      =      0.0          ! Coupling constant (isovector) of the J^2 terms
Vn       =  -1250.00000      ! Pairing strength (neutrons) of the ULB interaction.
Vp       =  -1250.00000      ! Pairing strength (protons) of the ULB interaction.
alpha    =    1.0            ! Alpha parameter in the ULB pairing interaction.
sigma_p  =    1.0            ! Power of the density dependence in the pairing
                            ! interaction. Alpha in EQ. (2) in
                            !   S. Goriely et al, PRC88, 061302 (2013).
hbm(1)   = 20.73551910, 20.73551910  ! Value of hbar^2/2m
COM1Body = 2                ! Treatment of the one-body COM correction.
                            !   (0) => None.
                            !   (1) => Perturbative.
                            !   (2) => Self-consistent
COM2Body = 0                ! Treatment of the two-body COM correction.
                            !   (0) => None.
                            !   (1) => Perturbative.
CoulombTreatment = 1        ! Treatment of Coulomb interaction.
                            !   (0) => No Coulomb included.
                            !   (1) => Direct and exchange (Slater) contribution
                            !   (2) => Only direct contribution.
protonsize  =  0.0, 0.0     ! Rms radii  (r+, r-) of a double Gaussian model
                            ! to be used for the proton.
                            ! If 0.0, no folding is used for either Gaussian.
neutronsize =  0.0, 0.0     ! Rms radii  (r+, r-) of a double Gaussian model
                            ! to be used for the neutron.
                            ! If 0.0, no folding is used for either Gaussian.
                            ! Note that input for protons and neutrons is NOT analogous.
                            ! The input for protons is the proton rms-radius, while
                            ! for neutrons is r_+^2 , r_-^2 in the model of
                            ! Chandra & Sauer PRC13, 245
HOCOMform   = .false.       ! Whether or not to correct the above rms radii
                            ! for the c.m. correction in a HO basis.
nucleonsize_selfconsistent = .true./
                            ! Whether to use the folding self-consistently,
                            ! or only in the calculation of the energy.
rotcorr     = 0             ! Whether (1) or not (0) to include the rotational
                            ! correction for the energy based on the Belyaev
                            ! moment of intertia.
rotcorrb    = 0             ! Parameters b and c for the contribution of the
rotcorrc    = 0             ! rotational correction, based on Eq. (14)
                            ! from D.  Pena-Arteaga et al. EPJA 52, 320 (2016).
                            ! except for an extra minus sign, of which I'm
                            ! pretty sure it is a typo.
```

FIG. 2. Example of a .param file, corresponding to a calculation with the SLy4 parameterization. This input file corresponds to the NLO.func functional file.

IV. OUTPUT FOR THE BRUSSELS FITTING CODES

Output for the Brussels fitting codes is activated by entering a non-empty string for the `BXLFIT` parameter. The end-result is that Tantalus opens a file named `$BXLFITz$Zn$N.out` and writes a single line to it at the end of the iterations.

```
N, Z, Total energy, Q20(t), Q22(t), Q(t), Gamma(t), R_charge^RMS, Rotcorrection, iter, iomsg
```

The individual entries are

```
 N           = Average number of neutrons
 Z           = Average number of protons
Total Energy = Total energy, includes evereything including the rotational
               correction if asked for.
Q2M(t)       = Value of <Q_{2m}> in fm^2 for the total density.
               Note that these values depend on the orientation of the nucleus
               in the box.
Q(t)         = Total quadrupole moments of the total density. Units of fm^2.
               This is always positive and does not discriminate between
               prolate and oblate shapes, but in return it does not depend on
               the orientation of the nucleus in the box.
Gamma(t)     = Triaxality angle of the total density in DEGREES. This quantity
               depends on the orientation of the nucleus in the box.
R_charge^RMS = RMS radius of the charge density. Includes the finite size
               correction of the protons and neutrons if asked for.
Rotcorrection= Energy associated with the rotational correction.
iter         = Number of SCF iterations performed by the code before exiting.
iomsg        = Output message by the code. Currently can be
               CONVERGED : the code exited because it judged convergence to be reached.
               MAXITER   : the code exited becauseit reached its maximum of iterations.
               More exit codes will undoubtedly be added.
```